Skeleton-Based American Sign Language Classification Using MediaPipe and Early Fusion STA-GCN

Jesreal D. Lustre, Bea Clarise E. Bacaling,
Angel Janette D. Taglucop, Gil John Rey A. Naldoza
BS - CS3A

1. Overview of the Project

This project develops an Isolated Sign Language Recognition (ISLR) system capable of taking short video clips of American Sign Language (ASL) signs and classifying which sign is being performed. The system operates entirely from skeleton keypoints, it never directly processes raw pixel data. Instead, it extracts the positions of body joints and finger joints from each video frame, preprocesses those coordinates, and feeds them into a deep learning model trained to distinguish between sign classes.

The dataset used is WLASL (World-Level American Sign Language), sourced from Kaggle. The raw dataset consists of approximately 12,000 short video clips covering 2,000 ASL glosses (words), with each video named by a unique ID and organized via a companion WLASL_v0.3.json label file.

The project evolved significantly over its development, from a simple BiLSTM on body-only keypoints, through multiple stages of model improvement, keypoint extraction upgrades, and training methodology changes. This document walks through every major decision and evolution in the order it occurred.

2. Objectives of the Project

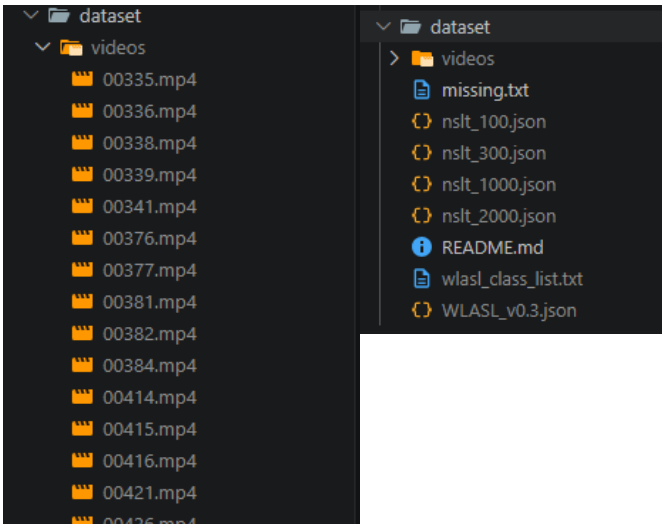
The project was designed to achieve the following goals:

- Build a pipeline that takes raw sign language videos as input and produces a classification of the sign being performed.
- Extract meaningful skeleton keypoints from video using computer vision tools (initially YOLO, then MediaPipe).
- Preprocess the raw keypoints into normalized, structured feature representations suitable for deep learning.
- Train a deep learning model capable of recognizing signs from skeleton sequences.
- Evaluate the model honestly using rigorous methodology to avoid overfitting to the evaluation set.
- Iteratively improve accuracy through better features, better model architectures, and better training strategies.

3. Data Collection

3.1 Dataset: WLASL (World-Level American Sign Language)

The raw dataset was downloaded from Kaggle (risangbaskoro/wlasl-processed). After download, the data was structured as a flat directory of approximately 12,000 .mp4 video files named by numeric ID, accompanied by a single WLASL_v0.3.json label file that maps each video ID to its corresponding ASL gloss (word label). No folder structure was provided to indicate class membership.



3.2 Dataset Organization

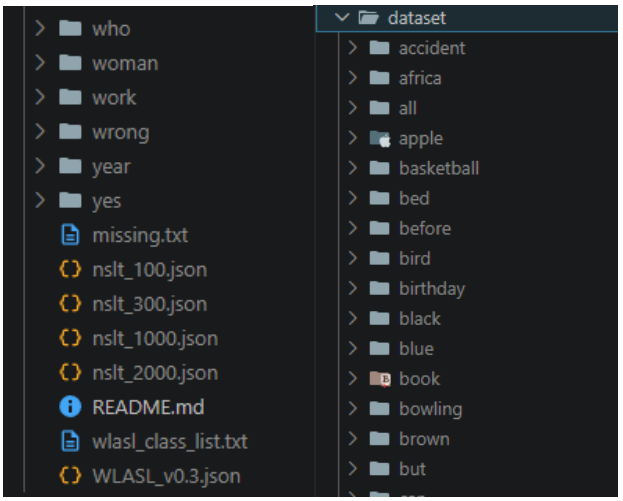
A preprocessing script (data/organize_dataset.py) was written to parse WLASL_v0.3.json, match each video ID to its gloss label, and copy videos into organized class subfolders under a dataset/ directory. A --subset N argument was used to limit training to the top N most-common glosses, which was critical for keeping processing time manageable:

```python
import os
import sys
import json
import shutil
import argparse
from tqdm import tqdm


# ── Config ──────────────────────────────────────────────────────────
ROOT        = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
JSON_PATH   = os.path.join(ROOT, "dataset", "WLASL_v0.3.json")
VIDEOS_DIR  = os.path.join(ROOT, "dataset", "videos")
DATASET_DIR = os.path.join(ROOT, "dataset")


def organize(subset=None, split=None, copy=True):

    # ── Load JSON ────────────────────────────────────────────────────
    if not os.path.exists(JSON_PATH):
        print(f"❌ JSON not found: {JSON_PATH}")
        print("   Make sure WLASL_v0.3.json is inside your data/ folder.")
        sys.exit(1)

    with open(JSON_PATH, "r") as f:
        data = json.load(f)

    print(f"✅ Loaded JSON: {len(data)} glosses total")

    # ── Apply subset (top-K glosses) ─────────────────────────────────
    if subset:
        data = data[:subset]
        print(f"   Using subset: top {subset} glosses (WLASL{subset})")

    # ── Stats tracking ───────────────────────────────────────────────
    copied   = 0
    skipped  = 0   # video file not found in data/videos/
    filtered = 0   # excluded by split filter

    os.makedirs(DATASET_DIR, exist_ok=True)

    # ── Main loop ────────────────────────────────────────────────────
    for entry in tqdm(data, desc="Organizing glosses"):
        gloss     = entry["gloss"]          # e.g. "hello"
        instances = entry["instances"]      # list of video instances

        gloss_dir = os.path.join(DATASET_DIR, gloss)
        os.makedirs(gloss_dir, exist_ok=True)

        for instance in instances:
            video_id    = str(instance["video_id"])
            inst_split  = instance.get("split", "train")

            # ── Split filter ─────────────────────────────────────────
            if split and inst_split != split:
                filtered += 1
                continue

            # ── Find source video ────────────────────────────────────
            src = os.path.join(VIDEOS_DIR, video_id + ".mp4")

            if not os.path.exists(src):
                skipped += 1
                continue

            # ── Destination ──────────────────────────────────────────
            dst = os.path.join(gloss_dir, video_id + ".mp4")

            if os.path.exists(dst):
                continue   # already organized, skip

            if copy:
                shutil.copy2(src, dst)
            else:
                shutil.move(src, dst)

            copied += 1

    # ── Summary ──────────────────────────────────────────────────────
    action = "Copied" if copy else "Moved"

    print(f"\n{'='*50}")
    print(f"  {action}   : {copied} videos")
    print(f"  Skipped  : {skipped} videos (file not found in data/videos/)")
    print(f"  Filtered : {filtered} videos (split filter applied)")
    print(f"{'='*50}")
    print(f"\n✅ Dataset organized → {DATASET_DIR}")

    # ── Show class distribution ──────────────────────────────────────
    classes = [
        d for d in os.listdir(DATASET_DIR)
        if os.path.isdir(os.path.join(DATASET_DIR, d))
    ]
    print(f"\n   Total classes : {len(classes)}")
```

```python
    # Show classes with video count
    class_counts = {}
    for c in classes:
        count = len([
            f for f in os.listdir(os.path.join(DATASET_DIR, c))
            if f.endswith(".mp4")
        ])
        class_counts[c] = count

    # Warn about classes with very few videos
    low = {k: v for k, v in class_counts.items() if v < 3}
    if low:
        print(f"\n⚠️  Classes with fewer than 3 videos (may hurt training):")
        for k, v in sorted(low.items(), key=lambda x: x[1]):
            print(f"     {k}: {v} video(s)")

    total_videos = sum(class_counts.values())
    print(f"\n   Total videos  : {total_videos}")
    print(f"   Avg per class : {total_videos / max(len(classes), 1):.1f}")

    # Save a summary JSON for reference
    summary = {
        "total_classes": len(classes),
        "total_videos" : total_videos,
        "class_counts" : class_counts
    }
    summary_path = os.path.join(ROOT, "data", "dataset_summary.json")
    with open(summary_path, "w") as f:
        json.dump(summary, f, indent=2)
    print(f"\n   Summary saved → {summary_path}")
```

```python
# ── Entry point ──────────────────────────────────────────────────────
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Organize WLASL flat videos into class subfolders"
    )
    parser.add_argument(
        "--subset", type=int, default=None,
        help="Use only the top-K glosses. E.g. --subset 100 for WLASL100"
    )
    parser.add_argument(
        "--split", type=str, default=None,
        choices=["train", "val", "test"],
        help="Only include videos from a specific split"
    )
    parser.add_argument(
        "--move", action="store_true",
        help="Move files instead of copying (faster but destructive)"
    )

    args = parser.parse_args()

    organize(
        subset = args.subset,
        split  = args.split,
        copy   = not args.move
    )
```

The organized dataset structure is shown below, with each folder corresponding to one ASL sign class. Experiments began with 20 classes and later expanded to 50 classes. One notable issue encountered was that the raw videos/ folder was inadvertently being picked up as a class, this was resolved by explicitly excluding it in the extraction script.



4. Data Preprocessing

Preprocessing underwent three major evolutionary phases, each motivated by limitations discovered in the previous approach.

4.1 Phase 1: YOLO-Based Extraction (Body Only, 34 Features)

The first extraction attempt used YOLOv8 pose (yolov8n-pose.pt). This was chosen due to a compatibility constraint: the project initially ran on Python 3.13, and MediaPipe only supports up to Python 3.11. YOLO was the practical workaround available.

YOLOv8 extracts 17 body keypoints in COCO format (× 2 coordinates = 34 features per frame). However, it has no dedicated hand landmark detection. For sign language, where hand shape and finger positions carry the majority of the linguistic signal, this representation proved wholly insufficient. The model achieved only 2–4% accuracy regardless of architecture improvements, confirming that the feature set, not the model, was the primary bottleneck.

4.2 Phase 2: YOLO Body + YOLO Hands (118 Features)

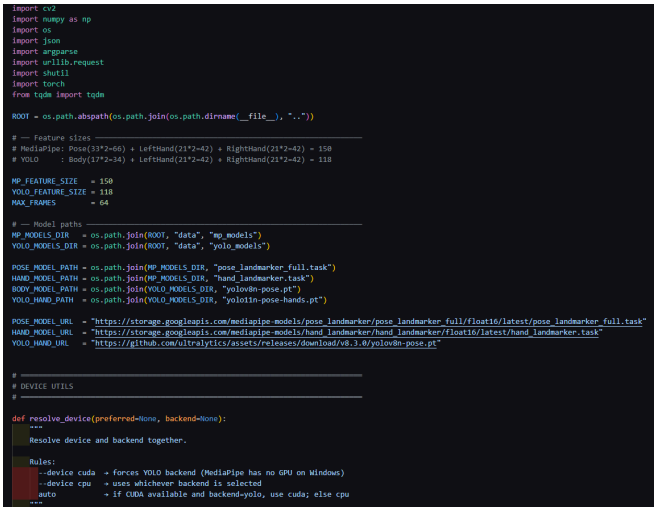
To improve hand coverage without abandoning YOLO, a second model dedicated to hand pose estimation was added: yolo11n-pose-hands.pt (21 keypoints per hand, sourced from GitHub). This extended the feature vector to 118 dimensions (34 body + 42 left hand + 42 right hand).

Despite the improved coverage, YOLO-based hand detection remained noisy on close-up sign language footage, where signers' hands often fill the frame in ways that differ from YOLO's general training distribution. Accuracy remained near 2%.

4.3 Phase 3: MediaPipe Tasks API (150 Features, Current Final Approach)

The decisive improvement came from switching to MediaPipe, which provides dedicated, high-accuracy models for both full-body pose estimation and hand landmark detection. To enable MediaPipe, the project was migrated from Python 3.13 to Python 3.11, and the virtual environment was rebuilt accordingly.

Two MediaPipe models were used: pose_landmarker_full.task (33 body keypoints) and hand_landmarker.task (21 keypoints per hand, both hands). This produced a 150-dimensional feature vector per frame: 33 pose × 2 + 21 left hand × 2 + 21 right hand × 2 = 150. Note that MediaPipe GPU acceleration is not supported on Windows, so all extraction was performed on CPU (AMD Ryzen 7 5700X3D), taking approximately 10–15 minutes for 20 classes.



```
import cv2
import numpy as np
import os
import json
import argparse
import urllib.request
import shutil
import torch
from tqdm import tqdm

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))

# ── Feature sizes ──────────────────────────────────────────────────────────
# MediaPipe: Pose(33*2=66) + LeftHand(21*2=42) + RightHand(21*2=42) = 150
# YOLO     : Body(17*2=34) + LeftHand(21*2=42) + RightHand(21*2=42) = 118

MP_FEATURE_SIZE   = 150
YOLO_FEATURE_SIZE = 118
MAX_FRAMES        = 64

# ── Model paths ────────────────────────────────────────────────────────────
MP_MODELS_DIR   = os.path.join(ROOT, "data", "mp_models")
YOLO_MODELS_DIR = os.path.join(ROOT, "data", "yolo_models")

POSE_MODEL_PATH = os.path.join(MP_MODELS_DIR, "pose_landmarker_full.task")
HAND_MODEL_PATH = os.path.join(MP_MODELS_DIR, "hand_landmarker.task")
BODY_MODEL_PATH = os.path.join(YOLO_MODELS_DIR, "yolov8n-pose.pt")
YOLO_HAND_PATH  = os.path.join(YOLO_MODELS_DIR, "yolo11n-pose-hands.pt")

POSE_MODEL_URL  = "https://storage.googleapis.com/mediapipe-models/pose_landmarker/pose_landmarker_full/float16/latest/pose_landmarker_full.task"
HAND_MODEL_URL  = "https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/latest/hand_landmarker.task"
YOLO_HAND_URL   = "https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n-pose.pt"


# ═══════════════════════════════════════════════════════════════════════════
# DEVICE UTILS
# ═══════════════════════════════════════════════════════════════════════════

def resolve_device(preferred=None, backend=None):
    """
    Resolve device and backend together.

    Rules:
      --device cuda  → forces YOLO backend (MediaPipe has no GPU on Windows)
      --device cpu   → uses whichever backend is selected
      auto           → if CUDA available and backend=yolo, use cuda; else cpu
    """
```

```
    if preferred == 'cuda':
        if not cuda_available:
            raise RuntimeError(
                "❌ CUDA requested but not available.\n"
                "   pip install torch torchvision "
                "--index-url https://download.pytorch.org/whl/cu121"
            )
        if backend == 'mediapipe':
            print("  ⚠️  MediaPipe has no GPU support on Windows.")
            print("  ⚠️  Switching backend to YOLO for CUDA support.")
            backend = 'yolo'
        return 'cuda', backend

    if preferred == 'cpu':
        return 'cpu', backend

    # Auto-detect
    if cuda_available and backend == 'yolo':
        print(f"  🚀 GPU detected: {torch.cuda.get_device_name(0)} — using CUDA")
        return 'cuda', backend

    return 'cpu', backend


# ═══════════════════════════════════════════════════════════════════════════
# MEDIAPIPE BACKEND
# ═══════════════════════════════════════════════════════════════════════════

def download_mp_models():
    """Download MediaPipe task model files if not present."""
    os.makedirs(MP_MODELS_DIR, exist_ok=True)
    for path, url, name in [
        (POSE_MODEL_PATH, POSE_MODEL_URL, "MP Pose model"),
        (HAND_MODEL_PATH, HAND_MODEL_URL, "MP Hand model"),
    ]:
        if not os.path.exists(path):
            print(f"  Downloading {name}...")
            urllib.request.urlretrieve(url, path)
            print(f"  ✅ {name} saved")
        else:
            print(f"  ✅ {name} already exists")


def make_mp_detectors():
    """Create MediaPipe pose and hand detectors."""
    import mediapipe as mp
    from mediapipe.tasks import python as mp_python
```

```
def make_mp_detectors():
    """Create MediaPipe pose and hand detectors."""
    import mediapipe as mp
    from mediapipe.tasks import python as mp_python
    from mediapipe.tasks.python.vision import (
        PoseLandmarker, PoseLandmarkerOptions,
        HandLandmarker, HandLandmarkerOptions
    )
    from mediapipe.tasks.python.vision.core.vision_task_running_mode import VisionTaskRunningMode

    pose_options = PoseLandmarkerOptions(
        base_options=mp_python.BaseOptions(model_asset_path=POSE_MODEL_PATH),
        running_mode=VisionTaskRunningMode.IMAGE,
        num_poses=1
    )
    hand_options = HandLandmarkerOptions(
        base_options=mp_python.BaseOptions(model_asset_path=HAND_MODEL_PATH),
        running_mode=VisionTaskRunningMode.IMAGE,
        num_hands=2
    )
    return (
        PoseLandmarker.create_from_options(pose_options),
        HandLandmarker.create_from_options(hand_options)
    )


def extract_mp(video_path, pose_det, hand_det, max_frames=MAX_FRAMES):
    """
    Extract skeleton using MediaPipe Tasks API.
    Returns: (max_frames, 150)
    """
    import mediapipe as mp

    cap = cv2.VideoCapture(video_path)
    keypoints = []

    while cap.isOpened() and len(keypoints) < max_frames:
        ret, frame = cap.read()
        if not ret:
            break

        rgb   = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        img   = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
        frame_kp = []

        # Pose (66)
        pose_res = pose_det.detect(img)
        if pose_res.pose_landmarks:
```

```python
def extract_mp(video_path, pose_det, hand_det, max_frames=MAX_FRAMES):
        pose_res = pose_det.detect(img)
        if pose_res.pose_landmarks:
            for lm in pose_res.pose_landmarks[0]:
                frame_kp.extend([lm.x, lm.y])
        else:
            frame_kp.extend([0.0] * 66)

        # Hands (84)
        left_kp  = [0.0] * 42
        right_kp = [0.0] * 42
        hand_res = hand_det.detect(img)
        if hand_res.hand_landmarks:
            for i, hand_lms in enumerate(hand_res.hand_landmarks):
                if i >= len(hand_res.handedness):
                    break
                side = hand_res.handedness[i][0].display_name
                kp   = [v for lm in hand_lms for v in [lm.x, lm.y]]
                if side == "Left":
                    left_kp = kp
                else:
                    right_kp = kp

        frame_kp.extend(left_kp)
        frame_kp.extend(right_kp)
        keypoints.append(frame_kp)

    cap.release()

    while len(keypoints) < max_frames:
        keypoints.append([0.0] * MP_FEATURE_SIZE)

    return np.array(keypoints, dtype=np.float32)


# ═══════════════════════════════════════════════════════════
# YOLO BACKEND
# ═══════════════════════════════════════════════════════════

def download_yolo_models():
    """Download YOLO body + hand models if not present."""
    from ultralytics import YOLO
    os.makedirs(YOLO_MODELS_DIR, exist_ok=True)

    if not os.path.exists(BODY_MODEL_PATH):
        print("  Downloading YOLO body model...")
        YOLO("yolov8n-pose.pt")
        if os.path.exists("yolov8n-pose.pt"):
```

```python
def download_yolo_models():
            shutil.move("yolov8n-pose.pt", BODY_MODEL_PATH)
        print(f"  ✅ YOLO body model saved")
    else:
        print(f"  ✅ YOLO body model already exists")

    if not os.path.exists(YOLO_HAND_PATH):
        print("  Downloading YOLO hand model...")
        try:
            urllib.request.urlretrieve(YOLO_HAND_URL, YOLO_HAND_PATH)
            print(f"  ✅ YOLO hand model saved")
        except Exception as e:
            print(f"  ⚠️  Hand model download failed: {e}")
            print("     Hand keypoints will be zeros.")
    else:
        print(f"  ✅ YOLO hand model already exists")


def make_yolo_models():
    """Load YOLO body and hand models."""
    from ultralytics import YOLO
    body = YOLO(BODY_MODEL_PATH)
    hand = YOLO(YOLO_HAND_PATH) if os.path.exists(YOLO_HAND_PATH) else None
    return body, hand


def extract_yolo(video_path, body_model, hand_model, device='cpu', max_frames=MAX_FRAMES):
    """
    Extract skeleton using YOLO pose models.
    Returns: (max_frames, 118)
    """
    cap = cv2.VideoCapture(video_path)
    keypoints = []

    while cap.isOpened() and len(keypoints) < max_frames:
        ret, frame = cap.read()
        if not ret:
            break

        frame_kp = []

        # Body (34)
        body_res = body_model(frame, verbose=False, device=device)
        body_kp  = [0.0] * 34
        if body_res and body_res[0].keypoints is not None:
            kps = body_res[0].keypoints.xy
            if len(kps) > 0 and kps[0].shape[0] == 17:
                body_kp = kps[0].cpu().numpy().flatten().tolist()
```

```python
def extract_yolo(video_path, body_model, hand_model, device='cpu', max_frames=MAX_FRAMES):
            if len(kps) > 0 and kps[0].shape[0] == 17:
                body_kp = kps[0].cpu().numpy().flatten().tolist()
        frame_kp.extend(body_kp)

        # Hands (84)
        left_kp  = [0.0] * 42
        right_kp = [0.0] * 42
        if hand_model is not None:
            hand_res = hand_model(frame, verbose=False, device=device)
            if hand_res and hand_res[0].keypoints is not None:
                detected = []
                for i in range(len(hand_res[0].keypoints)):
                    kps = hand_res[0].keypoints.xy[i].cpu().numpy()
                    if kps.shape[0] == 21:
                        detected.append((kps[0][0], kps.flatten().tolist()))
                detected.sort(key=lambda h: h[0])
                if len(detected) >= 1:
                    right_kp = detected[0][1]
                if len(detected) >= 2:
                    left_kp  = detected[1][1]

        frame_kp.extend(right_kp)
        frame_kp.extend(left_kp)
        keypoints.append(frame_kp)

    cap.release()

    while len(keypoints) < max_frames:
        keypoints.append([0.0] * YOLO_FEATURE_SIZE)

    return np.array(keypoints, dtype=np.float32)


# ═══════════════════════════════════════════════════════════
# MAIN DATASET BUILDER
# ═══════════════════════════════════════════════════════════

def build_dataset(data_dir, save_dir, backend='mediapipe', device='cpu',
                  max_frames=MAX_FRAMES, subset=None):

    os.makedirs(save_dir, exist_ok=True)
    feature_size = MP_FEATURE_SIZE if backend == 'mediapipe' else YOLO_FEATURE_SIZE

    # ── Setup backend ──────────────────────────────────────
    print(f"\nSetting up {backend.upper()} backend...")
    if backend == 'mediapipe':
        download_mp_models()
```

```python
def build_dataset(data_dir, save_dir, backend='mediapipe', device='cpu',
            pose_det, hand_det = make_mp_detectors()
            extractfn = lambda vp: extract_mp(vp, pose_det, hand_det, max_frames)
        else:
            download_yolo_models()
            body_model, hand_model = make_yolo_models()
            extractfn = lambda vp: extract_yolo(vp, body_model, hand_model, device, max_frames)

        # ── Scan class folders ─────────────────────────────
        EXCLUDE = {'videos', 'mp_models', 'yolo_models'}
        class_folders = sorted([
            f for f in os.listdir(data_dir)
            if os.path.isdir(os.path.join(data_dir, f))
            and f.lower() not in EXCLUDE
        ])

        if subset:
            class_folders = class_folders[:subset]

        if len(class_folders) == 0:
            print(f"❌ No class folders found. Run organize_dataset.py first.")
            return None, None, None

        print(f"\n  Backend  : {backend.upper()}")
        print(f"  Device   : {device.upper()}")
        print(f"  Features : {feature_size} per frame")
        print(f"  Classes  : {len(class_folders)}")
        print(f"  Frames   : {max_frames} per video\n")

        label_map = {}
        data      = []
        labels    = []

        for idx, label in enumerate(class_folders):
            folder_path = os.path.join(data_dir, label)
            label_map[label] = idx

            video_files = [
                f for f in os.listdir(folder_path)
                if f.lower().endswith(('.mp4', '.avi', '.mov', '.mkv'))
            ]

            print(f"[{idx+1}/{len(class_folders)}] '{label}' — {len(video_files)} videos")

            for vid_file in tqdm(video_files, leave=False):
                vid_path = os.path.join(folder_path, vid_file)
                try:
                    skeleton = extractfn(vid_path)
                    data.append(skeleton)
```

```python
def build_dataset(data_dir, save_dir, backend='mediapipe', device='cpu',
                labels.append(idx)
            except Exception as e:
                print(f"  [ERROR] Skipped {vid_file}: {e}")

    # ── Cleanup ──────────────────────────────────────────────────────
    if backend == 'mediapipe':
        pose_det.close()
        hand_det.close()

    # ── Save ─────────────────────────────────────────────────────────
    X = np.array(data,   dtype=np.float32)
    y = np.array(labels, dtype=np.int64)

    np.save(os.path.join(save_dir, "X_raw.npy"), X)
    np.save(os.path.join(save_dir, "y.npy"),     y)

    with open(os.path.join(save_dir, "label_map.json"), "w") as f:
        json.dump(label_map, f, indent=2)

    print(f"\n✅ Done!")
    print(f"   Backend  : {backend.upper()}")
    print(f"   Device   : {device.upper()}")
    print(f"   X shape  : {X.shape}  (samples, frames, {feature_size} features)")
    print(f"   y shape  : {y.shape}")
    print(f"   Classes  : {len(label_map)}")

    return X, y, label_map


# ═══════════════════════════════════════════════════════════════════
# CLI
# ═══════════════════════════════════════════════════════════════════

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Extract skeleton keypoints from sign language videos",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Examples:
  python preprocessing/extract.py                       # mediapipe, all classes
  python preprocessing/extract.py 20                    # mediapipe, first 20 classes
  python preprocessing/extract.py 20 --backend yolo     # yolo, first 20 classes
  python preprocessing/extract.py --device cuda         # auto-switch to yolo+GPU
  python preprocessing/extract.py 20 --backend yolo --device cuda
        """
    )
    parser.add_argument(
```

```
(.venv) PS D:\SCHOOL\machine-learning-sign-language-re

Setting up MEDIAPIPE backend...
  ✅ MP Pose model already exists
  ✅ MP Hand model already exists
INFO: Created TensorFlow Lite XNNPACK delegate for CPU
W0000 00:00:1775564892.902800   18676 inference_feedba
W0000 00:00:1775564892.918880   18676 inference_feedba
W0000 00:00:1775564892.934604   13748 inference_feedba
W0000 00:00:1775564892.942113   22676 inference_feedba

  Backend  : MEDIAPIPE
  Device   : CPU
  Features : 150 per frame
  Classes  : 50
  Frames   : 64 per video

[1/50] 'accident' — 13 videos
  0%|
0000 00:00:1775564892.980069   23072 landmark_projecti
[2/50] 'africa' — 9 videos
[3/50] 'all' — 8 videos
[4/50] 'apple' — 11 videos
[5/50] 'basketball' — 12 videos
[6/50] 'bed' — 13 videos
[7/50] 'before' — 16 videos
[8/50] 'bird' — 10 videos
[9/50] 'birthday' — 6 videos
[10/50] 'black' — 10 videos
[11/50] 'blue' — 8 videos
[12/50] 'book' — 6 videos
[13/50] 'bowling' — 13 videos
[14/50] 'brown' — 8 videos
[15/50] 'but' — 7 videos
[16/50] 'can' — 9 videos
[17/50] 'candy' — 13 videos
[18/50] 'chair' — 7 videos
[19/50] 'change' — 12 videos
```

```python
    )
    parser.add_argument(
        "subset", type=int, nargs="?", default=None,
        help="Only process first N classes (e.g. 20)"
    )
    parser.add_argument(
        "--backend", type=str, default="mediapipe",
        choices=["mediapipe", "yolo"],
        help="Extraction backend (default: mediapipe)"
    )
    parser.add_argument(
        "--device", type=str, default=None,
        choices=["cuda", "cpu"],
        help="Device: cuda=GPU (yolo only), cpu=CPU. Default: auto"
    )
    parser.add_argument(
        "--frames", type=int, default=MAX_FRAMES,
        help=f"Max frames per video (default: {MAX_FRAMES})"
    )

    args = parser.parse_args()

    # Resolve device + backend together
    device, backend = resolve_device(args.device, args.backend)

    DATA_DIR = os.path.join(ROOT, "dataset")
    SAVE_DIR = os.path.join(ROOT, "output")

    build_dataset(
        DATA_DIR, SAVE_DIR,
        backend    = backend,
        device     = device,
        max_frames = args.frames,
        subset     = args.subset
    )
```

```
[21/50] 'city' — 9 videos
[22/50] 'clothes' — 5 videos
[23/50] 'color' — 8 videos
[24/50] 'computer' — 14 videos
[25/50] 'cook' — 8 videos
[26/50] 'cool' — 16 videos
[27/50] 'corn' — 12 videos
[28/50] 'cousin' — 14 videos
[29/50] 'cow' — 9 videos
[30/50] 'dance' — 7 videos
[31/50] 'dark' — 12 videos
[32/50] 'deaf' — 11 videos
[33/50] 'decide' — 9 videos
[34/50] 'doctor' — 10 videos
[35/50] 'dog' — 11 videos
[36/50] 'drink' — 15 videos
[37/50] 'eat' — 7 videos
[38/50] 'enjoy' — 8 videos
[39/50] 'family' — 11 videos
[40/50] 'fine' — 9 videos
[41/50] 'finish' — 9 videos
[42/50] 'fish' — 10 videos
[43/50] 'forget' — 7 videos
[44/50] 'full' — 10 videos
[45/50] 'give' — 10 videos
[46/50] 'go' — 15 videos
[47/50] 'graduate' — 10 videos
[48/50] 'hat' — 9 videos
[49/50] 'hearing' — 8 videos
[50/50] 'help' — 14 videos

✅ Done!
   Backend  : MEDIAPIPE
   Device   : CPU
   X shape  : (508, 64, 150)  (samples, frames, 150 features)
   y shape  : (508,)
   Classes  : 50
(.venv) PS D:\SCHOOL\machine-learning-sign-language-recognition-WSASL>
```

4.4 Normalization Pipeline (normalize.py)

Raw keypoints were processed through a seven-step normalization pipeline designed to make the representations invariant to signer position, scale, and camera distance.

Step 1: Missing Keypoint Interpolation

MediaPipe occasionally fails to detect keypoints (e.g., when a hand moves off-screen). Missing values (zeros) are filled via linear interpolation between neighboring frames to produce a continuous signal.

Step 2: Joint Filtering (150 → 102 Features)

Many of the 150 raw features contribute noise rather than signal for ASL recognition. Leg joints (landmarks 25–32) are entirely irrelevant for hand signing; most fine facial details are similarly unnecessary. The pipeline retains only 9 upper-body joints (nose, shoulders, elbows, wrists, hips) plus all 21 left-hand and 21 right-hand landmarks = 51 joints total, yielding 102 features. This reduction improved accuracy noticeably.

Step 3: Centering and Scale Normalization

All coordinates are centered by subtracting the midpoint between the left and right hip, making the skeleton position-invariant (the signer may appear anywhere on screen). Coordinates are then divided by the torso height (mid-hip to nose distance), making the skeleton scale-invariant across signers at different distances from the camera.

Step 4: Relative Hand Coordinates

All finger joint coordinates are expressed relative to their respective wrist. This makes hand shape location-independent, the model learns the configuration of the hand, not its absolute position in the frame.

Step 5: Temporal Smoothing

A Gaussian filter ($\sigma = 1.0$) is applied along the time axis for each joint to reduce frame-by-frame jitter inherent in MediaPipe detection.

Steps 6 & 7: Multi-Stream Feature Derivation

From the cleaned and normalized joint stream, three additional feature streams are derived to capture different aspects of signing motion:

Stream	Formula	File	Meaning
Joint Positions	Raw normalized coords	X_final.npy	Where joints are
Bone Vectors	Child joint − Parent joint	X_bones.npy	Relative limb orientations
Joint Motion	joint(t) − joint(t−1)	X_motion.npy	How joints move over time
Bone Motion	bone(t) − bone(t−1)	X_bone_motion.npy	How limb orientations change

```python
import numpy as np
import os
from scipy.ndimage import gaussian_filter1d

UPPER_BODY_JOINTS = [0, 11, 12, 13, 14, 15, 16, 23, 24]

UPPER_BODY_FEAT = []
for j in UPPER_BODY_JOINTS:
    UPPER_BODY_FEAT.extend([j * 2, j * 2 + 1])

HAND_FEAT    = list(range(66, 150))
KEEP_INDICES = UPPER_BODY_FEAT + HAND_FEAT  # 102 total

_F_NOSE = 0
_F_LSH  = 1
_F_RSH  = 2
_F_LHI  = 7
_F_RHI  = 8

FILT_LEFT_HAND_OFFSET  = 18
FILT_RIGHT_HAND_OFFSET = 60
HAND_JOINTS = 21

# Bone edges in filtered 51-joint space
# (parent, child) — bone vector = child - parent
BONE_EDGES = [
    (0,1),(0,2),(1,2),(1,3),(3,5),(2,4),(4,6),(1,7),(2,8),(7,8),
    (9,10),(10,11),(11,12),(12,13),
    (9,14),(14,15),(15,16),(16,17),
    (9,18),(18,19),(19,20),(20,21),
    (9,22),(22,23),(23,24),(24,25),
    (9,26),(26,27),(27,28),(28,29),
    (30,31),(31,32),(32,33),(33,34),
    (30,35),(35,36),(36,37),(37,38),
    (30,39),(39,40),(40,41),(41,42),
    (30,43),(43,44),(44,45),(45,46),
    (30,47),(47,48),(48,49),(49,50),
]


def clean_missing_keypoints(skeleton, strategy="interpolate"):
    skeleton = skeleton.copy()
    T        = skeleton.shape[0]
    valid    = [t for t in range(T) if not np.all(skeleton[t] == 0)]
    if len(valid) == 0:
        return skeleton
    if strategy == "repeat":
        last = skeleton[valid[0]]
```

```python
def clean_missing_keypoints(skeleton, strategy="interpolate"):
    if strategy == "repeat":
        last = skeleton[valid[0]]
        for t in range(T):
            if np.all(skeleton[t] == 0):
                skeleton[t] = last
            else:
                last = skeleton[t]
    elif strategy == "interpolate":
        for t in range(T):
            if np.all(skeleton[t] == 0):
                before = [v for v in valid if v < t]
                after  = [v for v in valid if v > t]
                if before and after:
                    t0, t1 = before[-1], after[0]
                    alpha  = (t - t0) / (t1 - t0)
                    skeleton[t] = (1-alpha)*skeleton[t0] + alpha*skeleton[t1]
                elif before:
                    skeleton[t] = skeleton[before[-1]]
                elif after:
                    skeleton[t] = skeleton[after[0]]
    return skeleton


def filter_joints(X):
    return X[:, :, KEEP_INDICES].astype(np.float32)


def make_relative_hands(frame):
    frame = frame.copy()
    lx = frame[FILT_LEFT_HAND_OFFSET]
    ly = frame[FILT_LEFT_HAND_OFFSET + 1]
    if not (lx == 0.0 and ly == 0.0):
        for j in range(HAND_JOINTS):
            frame[FILT_LEFT_HAND_OFFSET + j*2]     -= lx
            frame[FILT_LEFT_HAND_OFFSET + j*2 + 1] -= ly
    rx = frame[FILT_RIGHT_HAND_OFFSET]
    ry = frame[FILT_RIGHT_HAND_OFFSET + 1]
    if not (rx == 0.0 and ry == 0.0):
        for j in range(HAND_JOINTS):
            frame[FILT_RIGHT_HAND_OFFSET + j*2]     -= rx
            frame[FILT_RIGHT_HAND_OFFSET + j*2 + 1] -= ry
    return frame


def normalize_skeleton(skeleton):
    skeleton = skeleton.copy().astype(np.float32)
    for t in range(skeleton.shape[0]):
```

```python
def normalize_skeleton(skeleton):
    skeleton = skeleton.copy().astype(np.float32)
    for t in range(skeleton.shape[0]):
        frame = skeleton[t]
        lhx = frame[_F_LHI * 2];  lhy = frame[_F_LHI * 2 + 1]
        rhx = frame[_F_RHI * 2];  rhy = frame[_F_RHI * 2 + 1]
        if lhx == 0.0 and rhx == 0.0:
            cx = frame[_F_NOSE * 2];  cy = frame[_F_NOSE * 2 + 1]
        else:
            cx = (lhx + rhx) / 2.0;  cy = (lhy + rhy) / 2.0
        if cx == 0.0 and cy == 0.0:
            continue
        frame[0::2] -= cx
        frame[1::2] -= cy
        lsx = frame[_F_LSH * 2];  lsy = frame[_F_LSH * 2 + 1]
        rsx = frame[_F_RSH * 2];  rsy = frame[_F_RSH * 2 + 1]
        torso = np.linalg.norm(np.array([(lsx+rsx)/2, (lsy+rsy)/2]))
        if torso > 1e-6:
            frame /= torso
        frame      = make_relative_hands(frame)
        skeleton[t] = frame
    return skeleton


def normalize_dataset(X):
    return np.array([normalize_skeleton(X[i]) for i in range(len(X))], dtype=np.float32)


def smooth_skeleton(skeleton, sigma=1.0):
    return gaussian_filter1d(skeleton, sigma=sigma, axis=0).astype(np.float32)


def smooth_dataset(X, sigma=1.0):
    return np.array([smooth_skeleton(X[i], sigma) for i in range(len(X))], dtype=np.float32)


def compute_bone_vectors(X):
    """Bone = child_joint - parent_joint. Shape: (N, T, 102)"""
    N, T, F = X.shape
    V       = F // 2
    joints  = X.reshape(N, T, V, 2)
    bones   = np.zeros_like(joints)
    for parent, child in BONE_EDGES:
        if parent < V and child < V:
            bones[:, :, child, :] = joints[:, :, child, :] - joints[:, :, parent, :]
    return bones.reshape(N, T, F).astype(np.float32)
```

```python
def compute_motion(X):
    """Frame-to-frame delta. motion[t] = X[t+1] - X[t]. Shape: (N, T, F)"""
    motion         = np.zeros_like(X, dtype=np.float32)
    motion[:, :-1] = X[:, 1:] - X[:, :-1]
    return motion


if __name__ == "__main__":
    ROOT       = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
    OUTPUT_DIR = os.path.join(ROOT, "output")

    X_raw_path = os.path.join(OUTPUT_DIR, "X_raw.npy")
    if not os.path.exists(X_raw_path):
        print("❌ X_raw.npy not found. Run extract.py first.")
        exit(1)

    print("Loading X_raw.npy ...")
    X_raw = np.load(X_raw_path)
    print(f"   Shape: {X_raw.shape}")

    print("\nStep 1 — Cleaning missing keypoints ...")
    X_clean = np.array([clean_missing_keypoints(X_raw[i]) for i in range(len(X_raw))], dtype=np.float32)

    print("\nStep 2 — Filtering joints (150 → 102) ...")
    X_filt = filter_joints(X_clean)
    print(f"   Shape: {X_filt.shape}")

    print("\nStep 3 — Normalizing (center + scale + relative hands) ...")
    X_norm = normalize_dataset(X_filt)

    print("\nStep 4 — Temporal smoothing ...")
    X_smooth = smooth_dataset(X_norm, sigma=1.0)

    print("\nStep 5 — Computing bone vectors ...")
    X_bones = compute_bone_vectors(X_smooth)

    print("\nStep 6 — Computing joint motion ...")
    X_motion = compute_motion(X_smooth)

    print("\nStep 7 — Computing bone motion ...")
    X_bone_motion = compute_motion(X_bones)

    np.save(os.path.join(OUTPUT_DIR, "X_normalized.npy"),  X_smooth)
    np.save(os.path.join(OUTPUT_DIR, "X_bones.npy"),       X_bones)
    np.save(os.path.join(OUTPUT_DIR, "X_motion.npy"),      X_motion)
    np.save(os.path.join(OUTPUT_DIR, "X_bone_motion.npy"), X_bone_motion)
```

```python
    print(f"\n✅ Saved all 4 streams → {OUTPUT_DIR}")
    print(f"   X_normalized  : {X_smooth.shape}      (joint positions)")
    print(f"   X_bones       : {X_bones.shape}       (bone vectors)")
    print(f"   X_motion      : {X_motion.shape}      (joint motion Δ)")
    print(f"   X_bone_motion : {X_bone_motion.shape} (bone motion Δ)")
```

```
(.venv) PS D:\SCHOOL\machine-learning-sign-language-recognition-WSASL> python preprocessing/normalize.py
Loading X_raw.npy ...
   Shape: (508, 64, 150)

Step 1 — Cleaning missing keypoints ...

Step 2 — Filtering joints (150 → 102) ...
   Shape: (508, 64, 102)

Step 3 — Normalizing (center + scale + relative hands) ...

Step 4 — Temporal smoothing ...

Step 5 — Computing bone vectors ...

Step 6 — Computing joint motion ...

Step 7 — Computing bone motion ...

✅ Saved all 4 streams → D:\SCHOOL\machine-learning-sign-language-recognition-WSASL\output
   X_normalized  : (508, 64, 102)      (joint positions)
   X_bones       : (508, 64, 102)       (bone vectors)
   X_motion      : (508, 64, 102)      (joint motion Δ)
   X_bone_motion : (508, 64, 102) (bone motion Δ)
(.venv) PS D:\SCHOOL\machine-learning-sign-language-recognition-WSASL>
```

4.5 Resampling (resample.py)

Videos in the WLASL dataset vary in length from approximately 1 to 4 seconds. To produce fixed-length inputs for the neural network, each sequence is resampled to exactly 64 frames using linear interpolation. Short sequences are upsampled and long sequences are downsampled, ensuring all inputs share the same temporal dimension.

```python
import numpy as np
import os


def temporal_resample(skeleton, target_len=64):

    T = len(skeleton)

    if T == target_len:
        return skeleton

    indices = np.linspace(0, T - 1, target_len).astype(int)
    return skeleton[indices]


def resample_dataset(X, target_len=64):

    resampled = []

    for i in range(len(X)):
        resampled.append(temporal_resample(X[i], target_len=target_len))

    return np.array(resampled, dtype=np.float32)


# — Run directly ——————————————————————————————
if __name__ == "__main__":
    ROOT       = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
    OUTPUT_DIR = os.path.join(ROOT, "output")
    TARGET_LEN = 64

    # Try to load normalized first, fall back to raw
    norm_path = os.path.join(OUTPUT_DIR, "X_normalized.npy")
    raw_path  = os.path.join(OUTPUT_DIR, "X_raw.npy")

    if os.path.exists(norm_path):
        print(f"Loading X_normalized.npy ...")
        X = np.load(norm_path)
```

```python
    if os.path.exists(norm_path):
        print(f"Loading X_normalized.npy ...")
        X = np.load(norm_path)
    elif os.path.exists(raw_path):
        print(f"Loading X_raw.npy (normalized version not found) ...")
        X = np.load(raw_path)
    else:
        print("❌ No data found. Run extract.py (and optionally normalize.py) first.")
        exit(1)

    print(f"   Input shape : {X.shape}")

    print(f"\nResampling to {TARGET_LEN} frames ...")
    X_resampled = resample_dataset(X, target_len=TARGET_LEN)

    save_path = os.path.join(OUTPUT_DIR, "X_final.npy")
    np.save(save_path, X_resampled)

    print(f"\n✅ Saved resampled data → {save_path}")
    print(f"   Output shape : {X_resampled.shape}")
```

5. Modeling

The model architecture underwent three distinct generational improvements, driven by growing understanding of the task's spatial-temporal structure.

5.1 Generation 1: BiLSTM (Baseline)

The initial model was a Bidirectional Long Short-Term Memory (BiLSTM) network, a standard baseline for sequential data tasks. The architecture takes the skeleton sequence as a flat feature vector per frame and reads it both forward and backward.

Architecture: Input (batch, 64, features) → BiLSTM (128 hidden, 2 layers) → Dropout (0.5) → Fully Connected (256 → num_classes).

Performance with this model revealed a critical insight: the feature set matters more than the model architecture at this stage. Switching from YOLO body-only features (2% accuracy) to MediaPipe features (26.8% accuracy) represented a larger gain than any model improvement made during this phase.

The fundamental limitation of the BiLSTM approach is that it treats each skeleton frame as a flat vector, with no awareness of spatial relationships between joints. It cannot know that the wrist connects to the elbow, or that adjacent finger joints should move coherently. This structural information is critical for accurate sign classification.

5.2 Generation 2: Custom Multi-Stream ST-GCN

The second architecture was a Spatial-Temporal Graph Convolutional Network (ST-GCN), implemented from scratch following the approach described by Yan et al. (AAAI 2018). ST-GCN addresses the BiLSTM's key limitation by representing the skeleton as a graph where nodes are joints and edges are anatomical connections, allowing the model to propagate information along physiologically meaningful pathways.

Input shape: (batch, 2, 64, 51) — 2 channels (x, y), 64 frames, 51 joints — processed through 9 ST-GCN blocks followed by global average pooling and a fully connected classification head. Three independent ST-GCN branches were used to process joint positions, motion (temporal delta), and bone vectors, with their output logits summed (late fusion) before the final softmax.

This architecture produced a test accuracy of 45.2% and a cross-validation mean of 62.4%, representing a substantial improvement over the BiLSTM baseline of 26.8%.

5.3 Generation 3: Ported Original ST-GCN with Enhancements (Current)

The final architecture ports the original ST-GCN implementation by Yan et al. (2018) to modern PyTorch, adding four significant enhancements informed by subsequent literature.

Enhancement 1: Bone Motion Stream (4th Stream)

Based on Miah et al. (2023), the temporal derivative of bone vectors (bone motion) was added as a fourth input stream. This stream distinguishes signs that share similar joint configurations but differ in movement speed or direction.

Enhancement 2: Early Fusion

In the previous generation, streams were combined by summing their final classification logits (late fusion), which prevented the model from learning cross-stream feature interactions. The current model instead concatenates 256-dimensional feature vectors from each stream into a 1024-dimensional representation, then passes this through a shared classification head, enabling the model to discover richer multi-stream relationships.

Enhancement 3: Adaptive Graph Topology

Inspired by Zhang et al. STA-GCN (2020), a learnable adjacency matrix is added on top of the fixed

anatomical graph. This allows the model to discover non-anatomical joint correlations — for example, both hands coordinating together in two-handed signs — that the fixed skeleton topology cannot represent.

Enhancement 4: DropGraph Regularization

Following Jiang et al. (2021), DropGraph randomly zeros entire joint channels during training, forcing the model to avoid relying on any single joint. This improves robustness to MediaPipe detection failures, which occasionally occur when hands partially leave the frame.

The full architecture is summarized below:

```
Four-Stream ST-GCN Architecture
├── Stream 1: Joint Positions  (N, 2, T, 51) → 10 ST-GCN layers → 256-d
├── Stream 2: Motion           (computed internally, 2nd-order diff)
├── Stream 3: Bone Vectors     (N, 2, T, 51) → 10 ST-GCN layers → 256-d
└── Stream 4: Bone Motion      (N, 2, T, 51) → 10 ST-GCN layers → 256-d

Early Fusion Head:
  concat(256, 256, 256, 256) = 1024-d
  → LayerNorm(1024)
  → FC(1024 → 512) → ReLU → Dropout(0.6)
  → FC(512 → num_classes)
```

```python
import torch
import torch.nn as nn

from models.st_gcn import Model as ST_GCN


class Model(nn.Module):

    def __init__(self, *args, early_fusion=True, **kwargs):
        super().__init__()

        self.early_fusion = early_fusion

        if early_fusion:
            # Each stream extracts 256-d features (no classifier head)
            kwargs_feat = {**kwargs, 'extract_features': True}
            self.joint_stream      = ST_GCN(*args, **kwargs_feat)
            self.motion_stream     = ST_GCN(*args, **kwargs_feat)
            self.bone_stream       = ST_GCN(*args, **kwargs_feat)
            self.bone_motion_stream= ST_GCN(*args, **kwargs_feat)

            # Early fusion classifier: 4 streams × 128 → num_class
            # (128-d per stream because st_gcn channels halved: 256→128)
            num_class = args[1]   # second positional arg
            self.fusion_fc = nn.Sequential(
                nn.Linear(128 * 4, 256),
                nn.LayerNorm(256),   # LayerNorm not BatchNorm — safe on batch size 1
                nn.ReLU(inplace=True),
                nn.Dropout(0.4),
                nn.Linear(256, num_class)
            )
        else:
            # Late fusion: each stream outputs logits, sum at end
            self.joint_stream      = ST_GCN(*args, **kwargs)
            self.motion_stream     = ST_GCN(*args, **kwargs)
            self.bone_stream       = ST_GCN(*args, **kwargs)
            self.bone_motion_stream= ST_GCN(*args, **kwargs)

```

```python
class Model(nn.Module):
    def __init__(self, *args, early_fusion=True, **kwargs):
            self.bone_motion_stream= ST_GCN(*args, **kwargs)

    def _compute_motion(self, x):
        N, C, T, V = x.size()
        zeros = torch.zeros(N, C, 1, V, device=x.device, dtype=x.dtype)
        return torch.cat([
            zeros,
            x[:, :, 1:-1] - 0.5 * x[:, :, 2:] - 0.5 * x[:, :, :-2],
            zeros
        ], dim=2)

    def forward(self, x, bone=None, bone_motion=None):
        # Compute second-order motion internally (original paper style)
        motion = self._compute_motion(x)

        # Fallback zeros if streams not provided
        if bone is None:
            bone = torch.zeros_like(x)
        if bone_motion is None:
            bone_motion = torch.zeros_like(x)

        if self.early_fusion:
            # Each stream → 256-d feature vector
            f_joint       = self.joint_stream(x)
            f_motion      = self.motion_stream(motion)
            f_bone        = self.bone_stream(bone)
            f_bone_motion = self.bone_motion_stream(bone_motion)

            # Concatenate all features (N, 1024)
            fused = torch.cat([f_joint, f_motion, f_bone, f_bone_motion], dim=1)

            # Shared classifier (allows cross-stream interaction)
            return self.fusion_fc(fused)

        else:
            # Late fusion: sum of logits
            return (
```

```python
        else:
            # Late fusion: sum of logits
            return (
                self.joint_stream(x) +
                self.motion_stream(motion) +
                self.bone_stream(bone) +
                self.bone_motion_stream(bone_motion)
            )
```

```python
import numpy as np
import torch
import torch.nn as nn


class Graph():

    def __init__(self, layout='mediapipe_51', strategy='spatial',
                 max_hop=1, dilation=1):
        self.max_hop = max_hop
        self.dilation = dilation
        self.get_edge(layout)
        self.hop_dis = get_hop_distance(self.num_node, self.edge, max_hop=max_hop)
        self.get_adjacency(strategy)

    def __str__(self):
        return str(self.A)

    def get_edge(self, layout):
        if layout == 'openpose':
            self.num_node = 18
            self_link     = [(i, i) for i in range(self.num_node)]
            neighbor_link = [
                (4,3),(3,2),(7,6),(6,5),(13,12),(12,11),(10,9),(9,8),
                (11,5),(8,2),(5,1),(2,1),(0,1),(15,0),(14,0),(17,15),(16,14)
            ]
            self.edge   = self_link + neighbor_link
            self.center = 1

        elif layout == 'ntu-rgb+d':
            self.num_node = 25
            self_link     = [(i, i) for i in range(self.num_node)]
            neighbor_1base = [
                (1,2),(2,21),(3,21),(4,3),(5,21),(6,5),(7,6),(8,7),(9,21),
                (10,9),(11,10),(12,11),(13,1),(14,13),(15,14),(16,15),(17,1),
                (18,17),(19,18),(20,19),(22,23),(23,8),(24,25),(25,12)
            ]
            neighbor_link = [(i-1, j-1) for (i,j) in neighbor_1base]
            self.edge   = self_link + neighbor_link
```

```python
import numpy as np
import torch
import torch.nn as nn


class Graph():

    def __init__(self, layout='mediapipe_51', strategy='spatial',
                 max_hop=1, dilation=1):
        self.max_hop = max_hop
        self.dilation = dilation
        self.get_edge(layout)
        self.hop_dis = get_hop_distance(self.num_node, self.edge, max_hop=max_hop)
        self.get_adjacency(strategy)

    def __str__(self):
        return str(self.A)

    def get_edge(self, layout):
        if layout == 'openpose':
            self.num_node = 18
            self_link     = [(i, i) for i in range(self.num_node)]
            neighbor_link = [
                (4,3),(3,2),(7,6),(6,5),(13,12),(12,11),(10,9),(9,8),
                (11,5),(8,2),(5,1),(2,1),(0,1),(15,0),(14,0),(17,15),(16,14)
            ]
            self.edge   = self_link + neighbor_link
            self.center = 1

        elif layout == 'ntu-rgb+d':
            self.num_node = 25
            self_link     = [(i, i) for i in range(self.num_node)]
            neighbor_1base = [
                (1,2),(2,21),(3,21),(4,3),(5,21),(6,5),(7,6),(8,7),(9,21),
                (10,9),(11,10),(12,11),(13,1),(14,13),(15,14),(16,15),(17,1),
                (18,17),(19,18),(20,19),(22,23),(23,8),(24,25),(25,12)
            ]
            neighbor_link = [(i-1, j-1) for (i,j) in neighbor_1base]
            self.edge   = self_link + neighbor_link
```

```python
class Graph():
    def get_edge(self, layout):
            self.center = 20

        elif layout == 'mediapipe_51':
            self.num_node = 51
            self_link     = [(i, i) for i in range(self.num_node)]

            upper_body = [
                (0,1),(0,2),(1,2),(1,3),(3,5),(2,4),(4,6),(1,7),(2,8),(7,8),
            ]
            hand_raw = [
                (0,1),(1,2),(2,3),(3,4),
                (0,5),(5,6),(6,7),(7,8),
                (0,9),(9,10),(10,11),(11,12),
                (0,13),(13,14),(14,15),(15,16),
                (0,17),(17,18),(18,19),(19,20),
                (5,9),(9,13),(13,17),
            ]
            left_hand  = [(a+9,  b+9)  for a,b in hand_raw]
            right_hand = [(a+30, b+30) for a,b in hand_raw]
            cross      = [(5,9),(6,30)]

            self.edge   = self_link + upper_body + left_hand + right_hand + cross
            self.center = 0

        else:
            raise ValueError(f"Unknown layout: {layout}")

    def get_adjacency(self, strategy):
        valid_hop         = range(0, self.max_hop + 1, self.dilation)
        adjacency         = np.zeros((self.num_node, self.num_node))
        for hop in valid_hop:
            adjacency[self.hop_dis == hop] = 1
        normalize_adjacency = normalize_digraph(adjacency)

        if strategy == 'uniform':
            A    = np.zeros((1, self.num_node, self.num_node))
            A[0] = normalize_adjacency
```

```python
            self.A = A

        elif strategy == 'distance':
            A = np.zeros((len(valid_hop), self.num_node, self.num_node))
            for i, hop in enumerate(valid_hop):
                A[i][self.hop_dis == hop] = normalize_adjacency[self.hop_dis == hop]
            self.A = A

        elif strategy == 'spatial':
            A = []
            for hop in valid_hop:
                a_root    = np.zeros((self.num_node, self.num_node))
                a_close   = np.zeros((self.num_node, self.num_node))
                a_further = np.zeros((self.num_node, self.num_node))
                for i in range(self.num_node):
                    for j in range(self.num_node):
                        if self.hop_dis[j, i] == hop:
                            if self.hop_dis[j, self.center] == self.hop_dis[i, self.center]:
                                a_root[j, i] = normalize_adjacency[j, i]
                            elif self.hop_dis[j, self.center] > self.hop_dis[i, self.center]:
                                a_close[j, i] = normalize_adjacency[j, i]
                            else:
                                a_further[j, i] = normalize_adjacency[j, i]
                if hop == 0:
                    A.append(a_root)
                else:
                    A.append(a_root + a_close)
                    A.append(a_further)
            self.A = np.stack(A)

        else:
            raise ValueError(f"Unknown strategy: {strategy}")


# ═══════════════════════════════════════════════════════════════
# ADAPTIVE GRAPH TOPOLOGY
```

```python
# ADAPTIVE GRAPH TOPOLOGY
# ═══════════════════════════════════════════════════════════════

class AdaptiveAdjacency(nn.Module):

    def __init__(self, num_joints, num_subsets, alpha=0.1):
        super().__init__()
        self.alpha = alpha
        # Learnable adjacency: initialized small so physical graph dominates early
        self.A_adaptive = nn.Parameter(
            torch.zeros(num_subsets, num_joints, num_joints) * 0.01
        )

    def forward(self, A_fixed):
        """
        Args:
            A_fixed: (K, V, V) fixed adjacency from Graph()

        Returns:
            (K, V, V) combined adjacency
        """
        # Softmax over source dimension to keep weights normalized
        A_learn = torch.softmax(self.A_adaptive, dim=2)
        return A_fixed + self.alpha * A_learn


# ═══════════════════════════════════════════════════════════════
# DROPGRAPH REGULARIZATION
# ═══════════════════════════════════════════════════════════════

def drop_graph(x, drop_prob=0.1, training=True):

    if not training or drop_prob == 0.0:
        return x

    N, C, T, V = x.shape

    # Create a mask over joints: (N, 1, 1, V)
    keep_prob = 1.0 - drop_prob
```

```python
def drop_graph(x, drop_prob=0.1, training=True):
    # Create a mask over joints: (N, 1, 1, V)
    keep_prob = 1.0 - drop_prob
    mask      = torch.bernoulli(
        torch.full((N, 1, 1, V), keep_prob, device=x.device)
    )

    # Scale to maintain expected value
    return x * mask / keep_prob


# ═══════════════════════════════════════════════════════════════
# GRAPH UTILS
# ═══════════════════════════════════════════════════════════════

def get_hop_distance(num_node, edge, max_hop=1):
    A = np.zeros((num_node, num_node))
    for i, j in edge:
        A[j, i] = 1
        A[i, j] = 1
    hop_dis      = np.zeros((num_node, num_node)) + np.inf
    transfer_mat = [np.linalg.matrix_power(A, d) for d in range(max_hop + 1)]
    arrive_mat   = (np.stack(transfer_mat) > 0)
    for d in range(max_hop, -1, -1):
        hop_dis[arrive_mat[d]] = d
    return hop_dis


def normalize_digraph(A):
    Dl       = np.sum(A, 0)
    num_node = A.shape[0]
    Dn       = np.zeros((num_node, num_node))
    for i in range(num_node):
        if Dl[i] > 0:
            Dn[i, i] = Dl[i] ** (-1)
    return np.dot(A, Dn)


def normalize_undigraph(A):
```

```python


def normalize_undigraph(A):
    Dl       = np.sum(A, 0)
    num_node = A.shape[0]
    Dn       = np.zeros((num_node, num_node))
    for i in range(num_node):
        if Dl[i] > 0:
            Dn[i, i] = Dl[i] ** (-0.5)
    return np.dot(np.dot(Dn, A), Dn)
```

6. Evaluation

6.1 Evaluation Methodology

Phase 1: Simple Train/Validation/Test Split

Initially the dataset was partitioned 70/15/15 (train/validation/test). With only 201 samples across 20 classes, this approach proved fundamentally unreliable: different random seeds could shift test accuracy by 5–10 percentage points simply by placing harder samples in the test partition. Stratification was also required to ensure every class was represented across all splits.

Phase 2: Stratified K-Fold Cross-Validation (Current)

The evaluation was upgraded to Stratified K-Fold Cross-Validation (K=4). A fixed test set of 31 samples was held out from all folds. The remaining data was divided into 4 folds; in each iteration, the model was trained on 3 folds and validated on the 4th. K=4 was chosen over K=5 because some classes contain only 5–6 samples — with K=5, the stratification constraint becomes impossible to satisfy for minority classes.

The primary reported metric is CV Mean ± Standard Deviation across all folds (e.g., 0.624 ± 0.043), supplemented by the final test accuracy on the held-out set. The CV Mean is the more trustworthy figure: with only 31 test samples, a single misclassification shifts test accuracy by approximately 3 percentage points, whereas the cross-validation estimate averages over four independent validation sets.

```
Loading data streams...
  Joint stream  : X_final.npy
  Bone stream   : X_bones.npy
  Bone motion   : X_bone_motion.npy

  X shape  : (508, 64, 102)
  Classes  : 50 | min: 5 | max: 16 | mean: 10.2

  Classes: 50  Params: 3,591,758  K-Folds: 4
  Epochs: 300 (patience=35)  SWA from ep 50
  Dropout: 0.4  DropGraph: 0.1  wd: 5e-2

  Train+Val: 431  Test: 77
============================================================

— Fold 1 ——————————————————————————————
   Train: 323  Val: 108  Patience: 35  SWA from ep 50
  Ep   1/300 | Train Acc: 0.006 | Val Acc: 0.009  <- best | Gap: -0.003 | Patience: 0/35
  Ep  10/300 | Train Acc: 0.071 | Val Acc: 0.102  <- best | Gap: -0.031 | Patience: 0/35
  Ep  20/300 | Train Acc: 0.133 | Val Acc: 0.139 | Gap: -0.006 | Patience: 5/35
  Ep  30/300 | Train Acc: 0.149 | Val Acc: 0.194  <- best | Gap: -0.046 | Patience: 0/35
  Ep  40/300 | Train Acc: 0.207 | Val Acc: 0.231 | Gap: -0.024 | Patience: 1/35
  [SWA accumulation started at ep 50]
  Ep  50/300 | Train Acc: 0.300 | Val Acc: 0.222 | Gap: 0.078 | Patience: 11/35
  Ep  60/300 | Train Acc: 0.421 | Val Acc: 0.241 | Gap: 0.180 | Patience: 3/35
  Ep  70/300 | Train Acc: 0.495 | Val Acc: 0.315 | Gap: 0.181 | Patience: 13/35
  Ep  80/300 | Train Acc: 0.514 | Val Acc: 0.296 | Gap: 0.218 | Patience: 23/35
  Ep  90/300 | Train Acc: 0.573 | Val Acc: 0.269 | Gap: 0.304 | Patience: 33/35
  Early stopping at epoch 92
  Calibrating SWA BN stats (43 weight snapshots)...
  SWA val acc: 0.019  (base best: 0.324)
  Base model better — keeping base weights
  Best val acc: 0.324 (stopped at epoch 92)
  Fold 1 best val acc: 0.324
```

```
— Fold 2 ——————————————————————————————
   Train: 323  Val: 108  Patience: 35  SWA from ep 50
  Ep   1/300 | Train Acc: 0.022 | Val Acc: 0.009  <- best | Gap: 0.012 | Patience: 0/35
  Ep  10/300 | Train Acc: 0.077 | Val Acc: 0.102 | Gap: -0.024 | Patience: 1/35
  Ep  20/300 | Train Acc: 0.093 | Val Acc: 0.157 | Gap: -0.065 | Patience: 4/35
  Ep  30/300 | Train Acc: 0.192 | Val Acc: 0.204 | Gap: -0.012 | Patience: 5/35
  Ep  40/300 | Train Acc: 0.245 | Val Acc: 0.213 | Gap: 0.032 | Patience: 6/35
  [SWA accumulation started at ep 50]
  Ep  50/300 | Train Acc: 0.350 | Val Acc: 0.231 | Gap: 0.118 | Patience: 2/35
  Ep  60/300 | Train Acc: 0.433 | Val Acc: 0.296 | Gap: 0.137 | Patience: 12/35
  Ep  70/300 | Train Acc: 0.461 | Val Acc: 0.259 | Gap: 0.202 | Patience: 22/35
  Ep  80/300 | Train Acc: 0.526 | Val Acc: 0.296 | Gap: 0.230 | Patience: 3/35
  Ep  90/300 | Train Acc: 0.517 | Val Acc: 0.259 | Gap: 0.258 | Patience: 2/35
  Ep 100/300 | Train Acc: 0.514 | Val Acc: 0.296 | Gap: 0.218 | Patience: 12/35
  Ep 110/300 | Train Acc: 0.622 | Val Acc: 0.278 | Gap: 0.345 | Patience: 22/35
  Ep 120/300 | Train Acc: 0.622 | Val Acc: 0.324 | Gap: 0.298 | Patience: 32/35
  Early stopping at epoch 123
  Calibrating SWA BN stats (74 weight snapshots)...
  SWA val acc: 0.037  (base best: 0.343)
  Base model better — keeping base weights
  Best val acc: 0.343 (stopped at epoch 123)
  Fold 2 best val acc: 0.343

— Fold 3 ——————————————————————————————
   Train: 323  Val: 108  Patience: 35  SWA from ep 50
  Ep   1/300 | Train Acc: 0.015 | Val Acc: 0.019  <- best | Gap: -0.003 | Patience: 0/35
  Ep  10/300 | Train Acc: 0.071 | Val Acc: 0.130 | Gap: -0.058 | Patience: 1/35
  Ep  20/300 | Train Acc: 0.136 | Val Acc: 0.194  <- best | Gap: -0.058 | Patience: 0/35
  Ep  30/300 | Train Acc: 0.155 | Val Acc: 0.250  <- best | Gap: -0.095 | Patience: 0/35
  Ep  40/300 | Train Acc: 0.214 | Val Acc: 0.185 | Gap: 0.028 | Patience: 5/35
  [SWA accumulation started at ep 50]
  Ep  50/300 | Train Acc: 0.294 | Val Acc: 0.259 | Gap: 0.035 | Patience: 7/35
  Ep  60/300 | Train Acc: 0.443 | Val Acc: 0.324  <- best | Gap: 0.119 | Patience: 0/35
  Ep  70/300 | Train Acc: 0.406 | Val Acc: 0.324 | Gap: 0.081 | Patience: 10/35
  Ep  80/300 | Train Acc: 0.483 | Val Acc: 0.398  <- best | Gap: 0.085 | Patience: 0/35
  Ep  90/300 | Train Acc: 0.539 | Val Acc: 0.352 | Gap: 0.187 | Patience: 10/35
  Ep 100/300 | Train Acc: 0.567 | Val Acc: 0.287 | Gap: 0.280 | Patience: 20/35
  Ep 110/300 | Train Acc: 0.573 | Val Acc: 0.315 | Gap: 0.258 | Patience: 30/35
  Early stopping at epoch 115
  Calibrating SWA BN stats (66 weight snapshots)...
  SWA val acc: 0.028  (base best: 0.398)
  Base model better — keeping base weights
  Best val acc: 0.398 (stopped at epoch 115)
  Fold 3 best val acc: 0.398
```

```
— Fold 4 ——————————————————————————————
   Train: 324  Val: 107  Patience: 35  SWA from ep 50
  Ep   1/300 | Train Acc: 0.019 | Val Acc: 0.028  <- best | Gap: -0.010 | Patience: 0/35
  Ep  10/300 | Train Acc: 0.062 | Val Acc: 0.093 | Gap: -0.032 | Patience: 1/35
  Ep  20/300 | Train Acc: 0.114 | Val Acc: 0.187 | Gap: -0.073 | Patience: 2/35
  Ep  30/300 | Train Acc: 0.213 | Val Acc: 0.215 | Gap: -0.002 | Patience: 6/35
  Ep  40/300 | Train Acc: 0.228 | Val Acc: 0.206 | Gap: 0.023 | Patience: 16/35
  [SWA accumulation started at ep 50]
  Ep  50/300 | Train Acc: 0.312 | Val Acc: 0.243 | Gap: 0.069 | Patience: 7/35
  Ep  60/300 | Train Acc: 0.401 | Val Acc: 0.262 | Gap: 0.140 | Patience: 7/35
  Ep  70/300 | Train Acc: 0.475 | Val Acc: 0.262 | Gap: 0.214 | Patience: 1/35
  Ep  80/300 | Train Acc: 0.475 | Val Acc: 0.327  <- best | Gap: 0.148 | Patience: 0/35
  Ep  90/300 | Train Acc: 0.531 | Val Acc: 0.327 | Gap: 0.204 | Patience: 5/35
  Ep 100/300 | Train Acc: 0.552 | Val Acc: 0.318 | Gap: 0.235 | Patience: 3/35
  Ep 110/300 | Train Acc: 0.599 | Val Acc: 0.224 | Gap: 0.374 | Patience: 13/35
  Ep 120/300 | Train Acc: 0.574 | Val Acc: 0.336 | Gap: 0.238 | Patience: 23/35
  Ep 130/300 | Train Acc: 0.620 | Val Acc: 0.327 | Gap: 0.293 | Patience: 33/35
  Early stopping at epoch 132
  Calibrating SWA BN stats (83 weight snapshots)...
  SWA val acc: 0.019  (base best: 0.364)
  Base model better — keeping base weights
  Best val acc: 0.364 (stopped at epoch 132)
  Fold 4 best val acc: 0.364
```

6.2 Training Improvements

The following improvements were applied incrementally to stabilize and improve training:

Improvement	Rationale
Early Stopping (patience=20–35)	Prevents wasting epochs after validation plateau; reduces overfitting
Warmup + Cosine LR Schedule	Gentler learning rate start (10 warmup epochs), then smooth decay

AdamW Optimizer	Decoupled weight decay for more effective regularization
Label Smoothing (0.1–0.15)	Prevents overconfident predictions; uses soft targets instead of hard 1/0 labels
Weighted Loss	Inverse-frequency class weights compensate for class imbalance
Gradient Clipping (1.0)	Prevents exploding gradients on small batch sizes

6.3 Data Augmentation

The following augmentations were applied per sample during training to improve generalization:

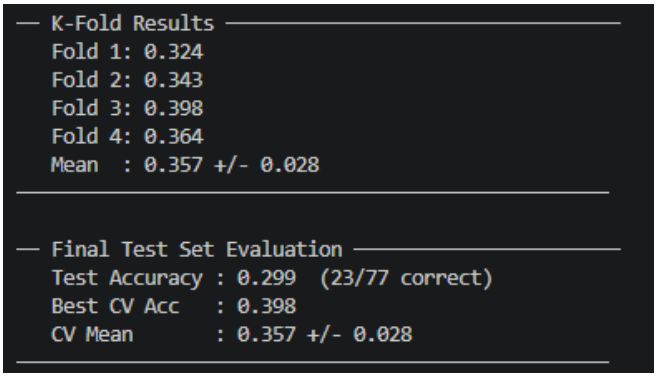
Augmentation	Effect
Gaussian Noise (σ=0.015)	Simulates natural jitter in MediaPipe keypoint detection
Random Scale (0.8–1.2×)	Simulates signers at different distances from the camera
Random Rotation (±20°)	Simulates minor variations in camera angle
Horizontal Flip	Doubles effective data size; models handedness invariance
Random Frame Drop (15%)	Simulates variable signing speed and partial occlusion
Time Warp	Stretches or compresses the temporal sequence non-linearly
Temporal Crop (75–100%)	Uses a random sub-window of the full sequence

6.4 Overfitting Diagnosis and Response

A persistent challenge throughout training was the gap between training and validation accuracy. The table below shows the evolution of this diagnostic across key runs:

Run	Train Acc.	Val Acc.	Gap	Status
Early runs	93%	45%	48%	Severe overfitting
After aggressive regularization	35%	37%	~0%	Underfitting
Target sweet spot	~65%	~55%	~10 %	Balanced

Severe overfitting was caused by approximately 13 million model parameters being trained on only 136 training samples per fold. Regularization strategies applied in response included increasing dropout from 0.4 to 0.6, increasing weight decay from 1e-3 to 5e-2, adding DropGraph regularization, and enforcing early stopping. When regularization was overly aggressive, the model underfitted; the optimal balance was found at dropout=0.5–0.6 with weight_decay=1e-2–5e-2.



6.5 Results Summary

The table below summarizes all model versions and their corresponding accuracy results. The current best result is highlighted.

Version	Model	Features	Test Acc.	CV Mean
BiLSTM (body only)	BiLSTM	34	2%	—
BiLSTM + YOLO Hands	BiLSTM	118	2%	—
BiLSTM + MediaPipe	BiLSTM	150	26.8%	—
Custom ST-GCN (filtered joints)	ST-GCN	102	33.3%	—
Multi-stream ST-GCN (3-stream)	Custom ST-GCN ×3	102×3	45.2%	62.4%
Original 2-stream ST-GCN (ported)	Yan et al. ×2	102×2	38.7%	51.8%
4-stream Early Fusion (current)	**Yan et al. ×4 + Enhancements**	**102×4**	**48.4%**	**51.2%**

Current best: 48.4% test accuracy / 51.2% CV Mean on 50 classes.

- Feature quality beats model complexity. The switch from YOLO (2%) to MediaPipe (26.8%) was the single largest accuracy gain in the entire project, larger than any model improvement. A better feature set outweighs a more sophisticated architecture when the input representation is poor.
- Data scarcity is the true bottleneck. With 6–16 samples per class, even a near-optimal model cannot generalize reliably. The literature recommends 50–100+ samples per class for stable training; the current 20-class experiment averages approximately 10 per class.
- Multi-stream input is highly effective. Each additional feature stream (bone, motion, bone motion) contributed a meaningful signal: expanding from 1 stream to 3 streams improved test accuracy from 33.3% to 45.2%.
- Late fusion vs. early fusion matters. Summing final classification logits (late fusion) prevents the model from learning cross-stream interactions. Early fusion via feature concatenation allows the fusion head to discover richer joint representations across streams.
- K-Fold cross-validation gives a more honest picture. A single train/test split on 201 samples is inherently noisy. K-Fold reveals not only average performance but also variance (±0.05), exposing when a model is unstable across different data partitions.
- More data is the next priority. The pipeline is now mature. Expanding from 20 to 50 classes gives approximately 500 total samples (~25 per class), which is where meaningful generalization begins to become achievable.

REFERENCES

Yan, S., Xiong, Y., & Lin, D. (2018). Spatial temporal graph convolutional networks for skeleton-based action recognition. *Proceedings of the AAAI Conference on Artificial Intelligence*, *32*(1), 7444–7452. https://doi.org/10.1609/aaai.v32i1.12328

Miah, A. S. M., Hasan, M. A. M., Shin, J., Okuyama, Y., & Tomioka, Y. (2023). Multistream spatial-temporal graph convolutional network for skeleton-based sign language recognition. *Sensors*, *23*(3), 1711. https://doi.org/10.3390/s23031711

Zhang, W., Lin, Z., Cheng, J., Ma, C., Deng, X., & Wang, H. (2020). STA-GCN: Two-stream graph convolutional network with spatial–temporal attention for hand gesture recognition. *The Visual Computer*, *36*(10–12), 2433–2444. https://doi.org/10.1007/s00371-020-01955-w

Jiang, S., Sun, B., Wang, L., Bai, Y., Li, K., & Fu, Y. (2021). *Skeleton Aware Multi-modal Sign Language Recognition* (arXiv:2103.08833). arXiv. https://doi.org/10.48550/arXiv.2103.08833

GitHub Repository Link:
https://github.com/JESREAL1JDL7LUSTRE/machine-learning-sign-language-recognition-WSASL.git

STA-GCN Repository:
https://github.com/yysijie/st-gcn.git

Datasets:
https://www.kaggle.com/datasets/risangbaskoro/wlasl-processed?resource=download&select=nslt_100.json